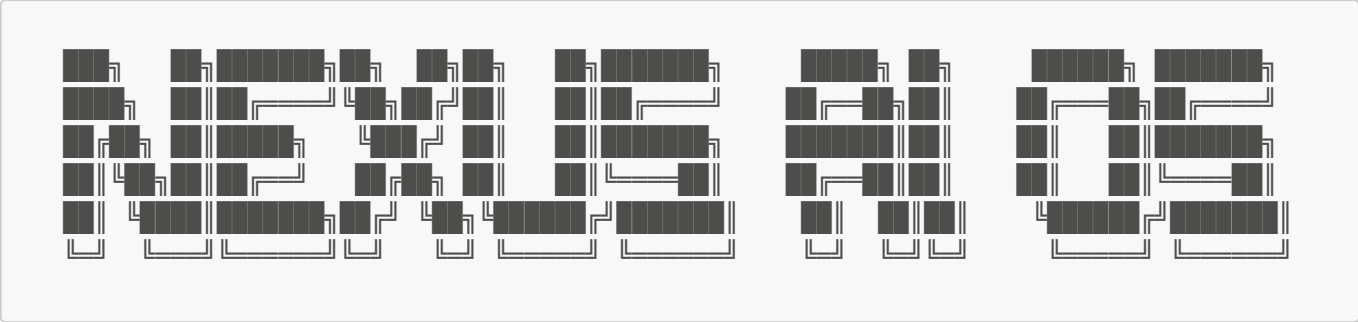

NEXUS AI OS

Your Personal AI Operating System — Local, Private, Intelligent

version 1.0.0 | license MIT | python 3.11+ | node 18+ | platform Windows | macOS | Linux | IoT ESP32 | AI Ollama

---

A futuristic, fully local AI operating system that orchestrates 13+ specialized agents for personal productivity, smart home automation, health tracking, financial management, and more — all running on your hardware with zero cloud dependency.

Quick Start • Features • Architecture • Documentation • Contributing

---

📋 Table of Contents

- Vision
- Features
- Architecture
- Screenshots
- System Requirements
- Quick Start
- Installation
  - Backend
  - Frontend
  - Mobile
  - ESP32
- Docker Deployment
- Configuration
- API Overview
- Agent System
- ESP32 Wiring
- Tech Stack
- Project Structure
- Contributing
- License
- Acknowledgments

🔭 Vision

NEXUS AI OS reimagines the personal computing experience by replacing fragmented apps with a unified AI-powered operating system. Every interaction flows through intelligent agents that understand your context, learn your preferences, and proactively assist you — all while keeping your data completely private on your own hardware.

Core Principles:

- 🔒 **Privacy First** — All AI inference runs locally via Ollama. Your data never leaves your machine.
- 🧠 **Continuous Learning** — The system learns your patterns, adapts to your style, and improves daily.
- 🏠 **IoT Native** — Control your entire smart home through natural language via ESP32 nodes.
- 🎙 **Voice Native** — Full speech-to-text and text-to-speech pipeline with wake-word detection.
- 📊 **Full Stack** — Desktop (Electron), Mobile (React Native), Backend (FastAPI), Firmware (ESP32).

✨ Features

#	Feature	Description
1	**AI Chat Interface**	Natural language conversations with context-aware AI across all domains
2	**Personal Companion**	Emotional support, personality learning, adaptive communication style
3	**Financial Manager**	Expense tracking, budget management, investment insights, financial reports
4	**Health & Wellness**	Physical/mental health tracking, fitness plans, nutrition, sleep analysis
5	**Smart Home Control**	ESP32-powered IoT: lights, fans, sensors, power monitoring, automation
6	**Voice Interface**	Wake-word activation, speech-to-text (Whisper), text-to-speech (TTS)
7	**Communication Hub**	Email management, message drafting, smart replies, inbox summarization
8	**Work Productivity**	Project management, code assistance, DevOps monitoring, meeting management
9	**Automated Reports**	PDF/Excel report generation for finance, health, home, and productivity
10	**Task Management**	Smart todo lists, priority-based scheduling, recurring tasks, reminders
11	**Workflow Automation**	Cron-based triggers, event-driven workflows, scheduled routines
12	**Security & Privacy**	Encrypted storage, secure authentication, threat detection, access control
13	**Memory & Learning**	Long-term memory with RAG, knowledge base, personality model fine-tuning

🏗 Architecture

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              NEXUS AI OS                                 │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐                │
│   │  Electron    │   │  React       │   │  React       │                │
│   │  Desktop App │   │  Web UI      │   │  Native App  │                │
│   │  (Windows/   │   │  (Vite +     │   │  (Mobile)    │                │
│   │   Mac/Linux) │   │   Tailwind)  │   │              │                │
│   └──────┬───────┘   └──────┬───────┘   └──────┬───────┘                │
│          │                  │                  │                         │
│          └──────────────────┴──────────────────┘                         │
│                             │                                            │
│                             │                                            │
│                   ┌─────────▼─────────┐                                  │
│                   │   WebSocket       │  Real-time communication         │
│                   │   + REST API      │  (FastAPI)                       │
│                   └─────────┬─────────┘                                  │
│                             │                                            │
│                             │                                            │
│          ┌──────────────────┼──────────────────┐                         │
│          │                  │                  │                         │
│   ┌──────▼──────┐  ┌───▼───┐  ┌──────▼──────┐                          │
│   │ Orchestrator │  │ Event │  │ Services    │                          │
│   │   Agent      │  │  Bus  │  │  Layer      │                          │
│   └──────┬──────┘  └───────┘  └─────────────┘                          │
│          │                                                               │
│     ┌────┴──────────────────────────────────────┐                        │
│     │          Specialized Agents               │                        │
│     │   ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐    │                        │
│     │   │Person│ │Financ│ │Health│ │ Home │    │                        │
│     │   │  al  │ │  ial │ │      │ │      │    │                        │
│     │   └──────┘ └──────┘ └──────┘ └──────┘    │                        │
│     │                                           │                        │
│     │   ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐    │                        │
│     │   │Voice │ │Comms │ │ Work │ │Report│    │                        │
│     │   └──────┘ └──────┘ └──────┘ └──────┘    │                        │
│     │                                           │                        │
│     │   ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐    │                        │
│     │   │Auto  │ │Learn │ │Memory│ │Secur │    │                        │
│     │   │mation│ │  ing │ │      │ │ity   │    │                        │
│     │   └──────┘ └──────┘ └──────┘ └──────┘    │                        │
│     └────┬──────────────────────────────────────┘                        │
│          │                                                               │
│     ┌────▼──────────────────────────────────────┐                        │
│     │            AI Pipeline                    │                        │
│     │  Ollama → LLM → RAG → Embeddings          │                        │
│     │  Whisper (STT) ← → TTS Engine             │                        │
│     └────┬──────────────────────────────────────┘                        │
│          │                                                               │
│     ┌────▼──────┐   ┌──────────┐   ┌──────────┐                         │
│     │  SQLite   │   │ ChromaDB │   │  MQTT    │                         │
│     │  Database │   │ VectorDB │   │  Broker  │                         │
│     └──────────┘   └──────────┘   └──────────┘                         │
│                                          │                               │
│                                          │                               │
│                                   ┌──────▼──────┐                        │
```

```
│                                │   ESP32 Nodes    │                  │
│                                │  (Sensors +      │                  │
│                                │   Actuators)     │                  │
│                                └──────────────────┘                  │
└──────────────────────────────────────────────────────────────────────┘
```

📸 Screenshots

> *Screenshots will be added after the first stable release.*

Dashboard	Chat Interface	Smart Home
Dashboard	Chat	Home

Health Tracking	Finance	Voice Control
Health	Finance	Voice

💻 System Requirements

Minimum Requirements

Component	Requirement
OS	Windows 10/11, macOS 12+, Ubuntu 20.04+
CPU	4 cores (8 recommended for local AI)
RAM	8 GB (16 GB recommended)
Storage	10 GB free (20 GB for AI models)
GPU	Optional but recommended for faster inference
Python	3.11 or higher
Node.js	18.x or higher
Ollama	Latest version

Optional

Component	Requirement
ESP32	ESP32-DevKitC or compatible board
MQTT Broker	Mosquitto (included in Docker setup)
PlatformIO	For ESP32 firmware development
Docker	For containerized deployment

🚀 Quick Start

Get NEXUS AI OS running in 5 steps:

```
# 1. Clone the repository
git clone https://github.com/nexus-ai/nexus-ai-os.git
cd nexus-ai-os

# 2. Install Ollama and pull the default model
# Visit https://ollama.ai to install Ollama, then:
ollama pull llama3.2

# 3. Run the setup script
# Windows:
.\scripts\setup.ps1
# macOS/Linux:
chmod +x scripts/setup.sh && ./scripts/setup.sh

# 4. Start everything
# Windows:
.\scripts\start.ps1
# macOS/Linux:
./scripts/start.sh

# 5. Open the app
# Desktop: http://localhost:5173
# API Docs: http://localhost:8000/docs
```

📦 Installation

Backend Setup

```
cd backend

# Create virtual environment
python -m venv venv

# Activate (Windows)
.\venv\Scripts\Activate.ps1
# Activate (macOS/Linux)
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Create .env file (optional — defaults work for development)
cp .env.example .env
```

```
# Start the backend server
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

The backend will be available at `http://localhost:8000`. API documentation at `http://localhost:8000/docs`.

Frontend Setup

```
cd frontend

# Install dependencies
npm install

# Development mode (browser)
npm run dev

# Development mode (Electron desktop app)
npm run electron:dev

# Production build
npm run build

# Build Electron app for distribution
npm run electron:build
```

The frontend development server runs on `http://localhost:5173`.

Mobile Setup

```
cd mobile

# Install dependencies
npm install

# Start Expo development server
npx expo start

# Run on Android
npx expo run:android

# Run on iOS (macOS only)
npx expo run:ios
```

ESP32 Setup

```
cd esp32

# Install PlatformIO (if not installed)
pip install platformio

# Edit WiFi/MQTT settings
# Open src/config.h and update:
#   WIFI_SSID_DEFAULT, WIFI_PASS_DEFAULT
#   MQTT_BROKER_DEFAULT

# Build firmware
pio run

# Upload to ESP32
pio run --target upload

# Monitor serial output
pio device monitor --baud 115200
```

> See docs/ESP32_SETUP.md for detailed hardware wiring and sensor calibration.

🐳 Docker Deployment

```
# Start all services
docker-compose up -d

# View logs
docker-compose logs -f

# Stop services
docker-compose down

# Rebuild after changes
docker-compose up -d --build
```

Services started by Docker:

Service	Port	Description
`backend`	8000	FastAPI backend server
`frontend`	80	React web UI (nginx)
`mosquitto`	1883 / 9001	MQTT broker
`ollama`	11434	Local AI inference server

⚙️ Configuration

NEXUS AI OS is configured through environment variables or a `.env` file in the `backend/` directory.

Core Settings

```
# Application
NEXUS_APP_NAME=NEXUS AI
NEXUS_VERSION=1.0.0
NEXUS_ENV=development
NEXUS_DEBUG=true
NEXUS_HOST=0.0.0.0
NEXUS_PORT=8000
NEXUS_SECRET_KEY=your-secret-key-here

# Database
NEXUS_DB_PATH=./data/nexus.db
NEXUS_VECTOR_DB_PATH=./data/chromadb
NEXUS_BACKUP_PATH=./data/backups

# Ollama
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=llama3.1
OLLAMA_EMBEDDING_MODEL=nomic-embed-text
OLLAMA_TIMEOUT=120

# MQTT
MQTT_BROKER_HOST=localhost
MQTT_BROKER_PORT=1883
MQTT_USERNAME=nexus
MQTT_PASSWORD=nexus_mqtt_password
MQTT_TOPIC_PREFIX=nexus/home

# Voice
VOICE_TTS_MODEL=tts_models/en/ljspeech/tacotron2-DDC
VOICE_STT_MODEL=base
VOICE_WAKE_WORD=nexus

# Email
EMAIL_SMTP_HOST=smtp.gmail.com
EMAIL_SMTP_PORT=587
EMAIL_USERNAME=
EMAIL_PASSWORD=

# User
USER_NAME=User
USER_TIMEZONE=UTC
USER_LANGUAGE=en

# Logging
LOG_LEVEL=DEBUG
LOG_FILE_PATH=./logs/nexus.log
```

📡 API Overview

NEXUS AI OS exposes a comprehensive REST API and WebSocket channels.

REST Endpoints

Prefix	Tag	Description
`POST /api/chat/message`	Chat	Send a message to the AI
`GET /api/chat/conversations`	Chat	List conversations
`GET /api/chat/conversations/{id}`	Chat	Get conversation details
`DELETE /api/chat/conversations/{id}`	Chat	Delete a conversation
`GET /api/agents/`	Agents	List all agents
`GET /api/agents/{name}`	Agents	Get agent details
`GET /api/agents/{name}/stats`	Agents	Get agent statistics
`POST /api/agents/{name}/message`	Agents	Message a specific agent
`GET /api/home/status`	Home	Get smart home status
`POST /api/home/devices/{id}/control`	Home	Control a device
`GET /api/home/rooms`	Home	List rooms
`GET /api/home/rooms/{room}`	Home	Get room details
`GET /api/home/energy`	Home	Get energy data
`POST /api/home/scenes`	Home	Activate a scene
`GET /api/home/sensors`	Home	Get sensor data
`POST /api/health/metrics`	Health	Log health metrics
`POST /api/health/mood`	Health	Log mood entry
`POST /api/health/workout`	Health	Log workout
`GET /api/health/dashboard`	Health	Get health dashboard
`GET /api/health/trends`	Health	Get health trends
`GET /api/health/goals`	Health	Get health goals
`POST /api/finance/transactions`	Finance	Add transaction
`GET /api/finance/summary`	Finance	Get financial summary
`GET /api/finance/budget`	Finance	Get budget overview
`POST /api/finance/goals`	Finance	Set financial goal
`GET /api/finance/trends`	Finance	Get spending trends

Prefix	Tag	Description
`GET /api/finance/insights`	Finance	Get financial insights
`POST /api/tasks/`	Tasks	Create a task
`GET /api/tasks/`	Tasks	List tasks
`PUT /api/tasks/{id}`	Tasks	Update a task
`DELETE /api/tasks/{id}`	Tasks	Delete a task
`GET /api/tasks/stats`	Tasks	Get task statistics
`GET /api/tasks/upcoming`	Tasks	Get upcoming tasks
`GET /api/system/status`	System	System health status
`GET /api/system/metrics`	System	System metrics
`GET /api/system/config`	System	Current configuration
`POST /api/system/backup`	System	Trigger backup
`GET /api/system/logs`	System	Get recent logs
`POST /api/voice/speak`	Voice	Text-to-speech
`POST /api/voice/listen`	Voice	Speech-to-text
`POST /api/voice/process`	Voice	Process voice command
`GET /api/voice/status`	Voice	Voice engine status
`POST /api/reports/generate`	Reports	Generate a report
`GET /api/reports/`	Reports	List reports
`GET /api/reports/{id}`	Reports	Get report details
`GET /api/reports/{id}/download`	Reports	Download report file

WebSocket Channels

Endpoint	Description
`ws://host/ws/chat`	Real-time AI chat streaming
`ws://host/ws/home`	IoT device state updates
`ws://host/ws/system`	System metrics streaming
`ws://host/ws/notifications`	Live notification feed

> Full API documentation available at `http://localhost:8000/docs` (Swagger) or `http://localhost:8000/redoc` (ReDoc).

See docs/API.md for detailed request/response examples.

🤖 Agent System

NEXUS AI uses an **Orchestrator Pattern** where a master agent routes incoming requests to specialized agents based on intent classification.

Agent	Purpose
Orchestrator	Routes requests, coordinates multi-agent workflows
Personal	Personal companion, emotional support, personality learning
Financial	Budgets, expenses, investments, financial advice
Health	Physical/mental health tracking, fitness, nutrition
Home	Smart home control, IoT devices, energy monitoring
Communication	Email management, message drafting, notifications
Voice	Speech-to-text, text-to-speech, voice commands
Work	Project management, code assistance, DevOps
Report	PDF/Excel report generation across all domains
Automation	Workflow automation, scheduled routines, triggers
Learning	Model fine-tuning, preference learning, adaptation
Memory	Long-term recall, knowledge base, context management
Security	Encryption, access control, threat detection
Task	Task management, reminders, priority scheduling

> See docs/AGENTS.md for detailed agent documentation.

🔌 ESP32 Wiring Diagram

```
                        ESP32-DevKitC V4
                  ┌──────────────────────┐
                  │         USB          │
                  │   ┌──────────────┐   │
                  │   │              │   │
     DHT22 ── GPIO4│              │GPIO13 ── Relay 1 (Light 1)
MQ-135 (A) ── GPIO34│              │GPIO12 ── Relay 2 (Light 2)
 MQ-2  (A) ── GPIO35│              │GPIO14 ── Relay 3 (Light 3)
HC-SR04 T  ── GPIO5 │    ESP32     │GPIO27 ── Relay 4 (Light 4)
HC-SR04 E  ── GPIO18│              │GPIO26 ── Fan Relay
PIR Motion ── GPIO19│              │GPIO25 ── Fan PWM
SCT-013(A) ── GPIO36│              │GPIO15 ── IR LED
Door Sw 1  ── GPIO21│              │GPIO2  ── Buzzer
Door Sw 2  ── GPIO22│              │GPIO23 ── NeoPixel Data
                  │   │              │   │
```

```
                        │    └──────────────┘    │
                        │        3V3   GND       │
                        └────────────────────────┘


  Sensors (Input)                     Actuators (Output)
  ├─ DHT22: Temperature/Humidity     ├─ 4-Ch Relay Module: Lights
  ├─ MQ-135: Air Quality (PPM)       ├─ Fan Relay + PWM Speed
  ├─ MQ-2: Gas Leak Detection        ├─ IR Blaster: AC Control
  ├─ HC-SR04: Water Tank Level       ├─ Buzzer: Alerts
  ├─ PIR: Motion Detection           └─ WS2812B NeoPixel: RGB Strip
  ├─ SCT-013: Power Monitoring
  └─ Reed Switches: Door/Window
```

> See docs/ESP32_SETUP.md for full setup instructions.

🛠 Tech Stack

Backend

Technology	Purpose
Python 3.11+	Core language
FastAPI	REST API + WebSocket framework
Uvicorn	ASGI server
SQLAlchemy 2.0	ORM & database abstraction
SQLite + aiosqlite	Primary database (async)
ChromaDB	Vector database for RAG
Ollama	Local LLM inference
LangChain	AI framework & RAG pipeline
Whisper	Speech-to-text
Coqui TTS	Text-to-speech
Paho MQTT	IoT communication
APScheduler	Task scheduling
ReportLab	PDF generation
Loguru	Logging

Frontend

Technology	Purpose
React 18	UI framework

Technology	Purpose
TypeScript	Type-safe JavaScript
Vite	Build tool & dev server
Tailwind CSS	Utility-first styling
Electron	Desktop app shell
Zustand	State management
React Query	Server state management
Recharts	Data visualization
Framer Motion	Animations
Radix UI	Accessible UI primitives
Lucide React	Icon library

Mobile

Technology	Purpose
React Native	Cross-platform mobile framework
Expo	Development & build toolchain
TypeScript	Type safety

IoT / Firmware

Technology	Purpose
ESP32	Microcontroller platform
PlatformIO	Build system & package manager
Arduino Framework	Firmware framework
PubSubClient	MQTT client library
ArduinoJson	JSON serialization
ESPAsyncWebServer	Embedded web server
DHT Sensor Library	Temperature/humidity sensor
IRremoteESP8266	Infrared remote control
NeoPixel	Addressable LED control

Infrastructure

Technology	Purpose

Technology	Purpose
Docker	Containerization
Docker Compose	Multi-container orchestration
Mosquitto	MQTT broker
Nginx	Static file serving

🗂 Project Structure

```
nexus-ai-os/
├── README.md                       # This file
├── docker-compose.yml              # Docker orchestration
├── Dockerfile.backend              # Backend container
├── Dockerfile.frontend             # Frontend container
├── .gitignore                      # Git ignore rules
│
├── backend/                        # Python FastAPI backend
│   ├── main.py                     # Application entry point
│   ├── requirements.txt            # Python dependencies
│   ├── agents/                     # AI agent system
│   │   ├── orchestrator.py         # Master agent router
│   │   ├── base_agent.py           # Abstract base agent
│   │   ├── personal_agent.py       # Personal companion
│   │   ├── financial_agent.py      # Financial management
│   │   ├── health_agent.py         # Health & wellness
│   │   ├── home_agent.py           # Smart home control
│   │   ├── communication_agent.py  # Email & messaging
│   │   ├── voice_agent.py          # Voice interface
│   │   ├── work_agent.py           # Work productivity
│   │   ├── report_agent.py         # Report generation
│   │   ├── automation_agent.py     # Workflow automation
│   │   ├── learning_agent.py       # Model learning
│   │   ├── memory_agent.py         # Long-term memory
│   │   ├── security_agent.py       # Security management
│   │   └── task_agent.py           # Task management
│   ├── api/                        # API layer
│   │   ├── routes/                 # REST endpoint handlers
│   │   ├── websocket.py            # WebSocket handlers
│   │   ├── middleware.py           # Request middleware
│   │   └── dependencies.py         # Dependency injection
│   ├── core/                       # Core infrastructure
│   │   ├── config.py               # Configuration management
│   │   ├── engine.py               # NEXUS engine (singleton)
│   │   ├── events.py               # Event bus system
│   │   ├── logger.py               # Logging setup
│   │   └── security.py             # Security utilities
│   ├── database/                   # Data access layer
│   │   ├── connection.py           # Database connection manager
│   │   ├── models.py               # SQLAlchemy models
```

```
│   │   └── repositories.py         # Data repositories
│   ├── models/                      # AI model layer
│   │   ├── local_llm.py             # Ollama LLM wrapper
│   │   ├── rag_engine.py            # RAG pipeline
│   │   ├── embeddings.py            # Embedding models
│   │   ├── personality.py           # Personality model
│   │   └── fine_tuner.py            # Model fine-tuning
│   ├── services/                    # Business logic services
│   │   ├── ai_service.py            # AI inference service
│   │   ├── voice_service.py         # Voice engine
│   │   ├── email_service.py         # Email management
│   │   ├── mqtt_service.py          # MQTT client
│   │   ├── scheduler_service.py     # Job scheduler
│   │   ├── notification_service.py # Notifications
│   │   ├── file_service.py          # File management
│   │   ├── training_service.py      # Model training
│   │   └── system_service.py        # System monitoring
│   ├── scheduler/                   # Scheduled jobs
│   │   ├── daily_routines.py        # Daily automation
│   │   ├── job_manager.py           # Job lifecycle
│   │   ├── triggers.py              # Event triggers
│   │   └── workflows.py            # Workflow engine
│   └── tests/                       # Test suite
│       ├── test_agents.py           # Agent unit tests
│       ├── test_services.py         # Service unit tests
│       └── test_api.py              # API integration tests
│
├── frontend/                        # Electron + React desktop app
│   ├── package.json                 # Node dependencies
│   ├── vite.config.ts               # Vite configuration
│   ├── tailwind.config.js           # Tailwind CSS config
│   ├── tsconfig.json                # TypeScript config
│   ├── index.html                   # HTML entry point
│   ├── electron/                    # Electron main process
│   │   ├── main.ts                  # Main process entry
│   │   ├── preload.ts               # Preload script
│   │   └── tray.ts                  # System tray
│   └── src/                         # React source code
│       ├── App.tsx                   # Root component
│       ├── main.tsx                  # React entry point
│       ├── pages/                   # Page components
│       ├── components/              # Reusable components
│       ├── hooks/                   # Custom React hooks
│       ├── lib/                     # Utilities & stores
│       ├── types/                   # TypeScript types
│       └── styles/                  # Global styles
│
├── mobile/                          # React Native mobile app
│   ├── App.tsx                      # Mobile entry point
│   ├── app.json                     # Expo configuration
│   ├── package.json                 # Node dependencies
│   └── src/                         # Mobile source code
│       ├── screens/                 # Screen components
│       ├── components/              # Reusable components
```

```
│       └── lib/                    # Utilities
│
├── esp32/                          # ESP32 firmware
│   ├── platformio.ini              # PlatformIO config
│   └── src/                        # Firmware source
│       ├── main.cpp                # Firmware entry point
│       ├── config.h                # Pin & MQTT config
│       ├── sensors.cpp/h           # Sensor drivers
│       ├── actuators.cpp/h         # Relay/LED control
│       ├── mqtt_handler.cpp/h      # MQTT communication
│       ├── power_monitor.cpp/h     # Energy monitoring
│       ├── wifi_manager.cpp/h      # WiFi management
│       ├── web_server.cpp/h        # Embedded web server
│       └── utils.cpp/h             # Helper functions
│
├── scripts/                        # Automation scripts
│   ├── setup.ps1 / setup.sh        # One-time setup
│   ├── start.ps1 / start.sh        # Start all services
│   └── train.ps1 / train.sh        # Trigger model training
│
└── docs/                           # Documentation
    ├── ARCHITECTURE.md             # System architecture
    ├── API.md                      # API reference
    ├── AGENTS.md                   # Agent documentation
    └── ESP32_SETUP.md              # ESP32 hardware guide
```

🤝 Contributing

We welcome contributions! Here's how you can help:

Getting Started

1. **Fork** the repository
2. **Clone** your fork: `git clone https://github.com/your-username/nexus-ai-os.git`
3. **Create a branch**: `git checkout -b feature/your-feature`
4. **Make changes** and write tests
5. **Commit**: `git commit -m "feat: add amazing feature"`
6. **Push**: `git push origin feature/your-feature`
7. **Open a Pull Request**

Commit Convention

We follow Conventional Commits:

- `feat:` — New feature
- `fix:` — Bug fix
- `docs:` — Documentation changes
- `style:` — Code style changes (formatting, etc.)
- `refactor:` — Code refactoring
- `test:` — Adding or updating tests

- `chore:` — Maintenance tasks

Code Style

- **Python**: Follow PEP 8, use type hints, docstrings for public functions
- **TypeScript**: ESLint + Prettier, functional components with hooks
- **C++**: Arduino-style, consistent naming with project conventions

Areas for Contribution

- 🐛 Bug fixes and issue resolution
- ✨ New agent capabilities
- 🌍 Internationalization (i18n)
- 📱 Mobile app features
- 🏠 New ESP32 sensor/actuator drivers
- 📚 Documentation improvements
- 🧪 Test coverage improvements
- 🎨 UI/UX enhancements

📄 License

This project is licensed under the **MIT License**.

```
MIT License

Copyright (c) 2025 NEXUS AI

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

🙏 Acknowledgments

NEXUS AI OS is built on the shoulders of incredible open-source projects:

- Ollama — Local LLM inference
- FastAPI — Modern Python web framework
- LangChain — AI application framework
- ChromaDB — Vector database
- OpenAI Whisper — Speech recognition
- Coqui TTS — Text-to-speech
- React — UI library
- Electron — Desktop app framework
- Tailwind CSS — Utility-first CSS
- PlatformIO — Embedded development platform
- Eclipse Mosquitto — MQTT broker

Built with 💚 for a smarter, more private future.

NEXUS AI OS — Your Intelligence, Your Hardware, Your Rules.